

Breast Cancer Image Classification Using Deep Learning

Professional project documentation for binary medical image classification

Project Overview

This project builds and evaluates deep learning models to classify breast cancer images into Cancer and Non-Cancer categories. It includes preprocessing, model training, transfer learning, model saving, prediction, and full evaluation using visual metrics.

Team Members

Mohamed Tarek
230104594
Mohab Badawy
230104613
Youssef Elshazly
230104603

Repository: <https://github.com/mohamedtarek750/Image-Processing-Project.git>

3	224 x 224	96.43%	DenseNet169
Models Compared	Input Image Size	Best Accuracy	Best Overall Model

1. Project Overview

The project focuses on the use of convolutional neural networks and transfer learning for breast cancer image classification. The main goal is to compare a custom Scratch CNN against two pretrained transfer learning models: ResNet50 and DenseNet169.

The classification task is binary: each uploaded or tested image is assigned to one of two classes: Cancer or Non-Cancer. The evaluation emphasizes not only accuracy but also precision, recall, F1-score, confusion matrix behavior, and ROC/AUC performance.

2. Problem Statement

In medical image classification, accuracy alone can be misleading. A model can have high accuracy while still failing to detect some cancer cases. For that reason, this project evaluates several complementary metrics, especially recall and F1-score, to better understand model reliability.

Important interpretation: For cancer detection, recall is especially important because false negatives mean actual cancer cases were missed. However, precision is also important because false positives can create unnecessary concern and additional testing.

3. Dataset and Preprocessing

The dataset was organized into training, validation, and testing folders. Each split contains two class folders: Cancer and Non-Cancer. Images were loaded from directories using Keras ImageDataGenerator.

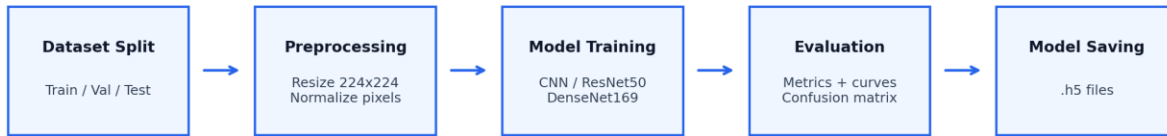
```
breast_cancer_split_train_balanced/  
|-- train/  
|   |-- Cancer/  
|   `-- Non-Cancer/  
|-- val/  
|   |-- Cancer/  
|   `-- Non-Cancer/  
`-- test/  
    |-- Cancer/  
    `-- Non-Cancer/
```

All images were resized to 224 x 224 pixels and normalized by rescaling pixel values from the range [0, 255] to [0, 1]. The training generator uses shuffle=True, while validation and testing use shuffle=False to keep prediction order aligned with ground-truth labels during evaluation.

```
data_generator = ImageDataGenerator(rescale=1/255)  
  
train_data = data_generator.flow_from_directory(  
    train_path,  
    target_size=(224, 224),  
    batch_size=16,  
    class_mode='categorical',  
    shuffle=True  
)  
  
test_data = data_generator.flow_from_directory(  
    test_path,
```

```
target_size=(224, 224),  
batch_size=16,  
class_mode='categorical',  
shuffle=False  
)
```

Project workflow:



4. Model Architectures

Three models were implemented to compare learning from scratch versus transfer learning.

4.1 Scratch CNN

The Scratch CNN learns features directly from the available breast cancer image dataset. It contains three convolutional blocks followed by dense classification layers. Dropout was added to reduce overfitting.

```
Input: 224 x 224 x 3
Conv2D(32, kernel_size=3, activation='relu') + MaxPooling2D
Conv2D(64, kernel_size=3, activation='relu') + MaxPooling2D
Conv2D(128, kernel_size=3, activation='relu') + MaxPooling2D
Flatten
Dense(128, activation='relu') + Dropout(0.5)
Dense(128, activation='relu') + Dropout(0.3)
Dense(2, activation='softmax')
```

4.2 ResNet50 Transfer Learning

ResNet50 was used with ImageNet pretrained weights. The convolutional base was frozen, and custom dense layers were added for the two-class breast cancer classification task.

4.3 DenseNet169 Transfer Learning

DenseNet169 was also used with ImageNet pretrained weights. Dense connections can improve feature reuse and gradient flow, which may help classification performance on image datasets.

Model	Feature Extractor	Trainable Base	Output Layer
Scratch CNN	Custom Conv2D blocks	Yes	Dense(2, softmax)
ResNet50	ImageNet pretrained ResNet50	No, frozen	Dense(2, softmax)
DenseNet169	ImageNet pretrained DenseNet169	No, frozen	Dense(2, softmax)

5. Training Setup

Each model was trained for 10 epochs using the Adam optimizer and categorical cross-entropy loss. The models used two output neurons with softmax activation, matching the categorical class mode used by the data generators.

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model_history = model.fit(
    train_data,
    epochs=10,
    validation_data=val_data
)
```

6. Evaluation Metrics

The evaluation stage is a core part of the project. The following metrics were used to evaluate the final models:

Metric	Meaning	Why It Matters
Accuracy	Overall correct prediction rate	Useful as a general performance indicator.
Precision	Positive predictive value	Important when false positives should be minimized.
Recall	True positive detection rate	Important in cancer detection because missed cancer cases are costly.
F1-Score	Harmonic mean of precision and recall	Useful when balancing false positives and false negatives.
Confusion Matrix	Class-by-class classification breakdown	Shows how many samples were correctly or incorrectly classified per class.
ROC/AUC	Binary classification threshold performance	Shows how well the model separates the two classes.
Loss and Accuracy Curves	Training vs. validation behavior across epochs	Helps detect overfitting or underfitting.

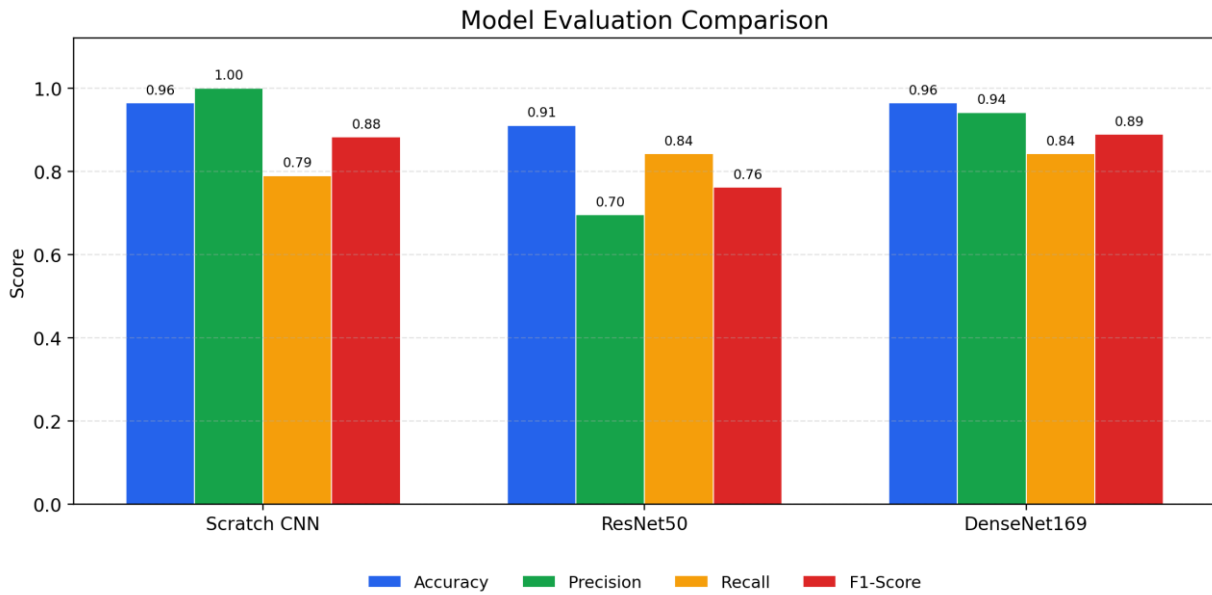
7. Results

The final model evaluation results are shown below. All values are calculated on the test set.

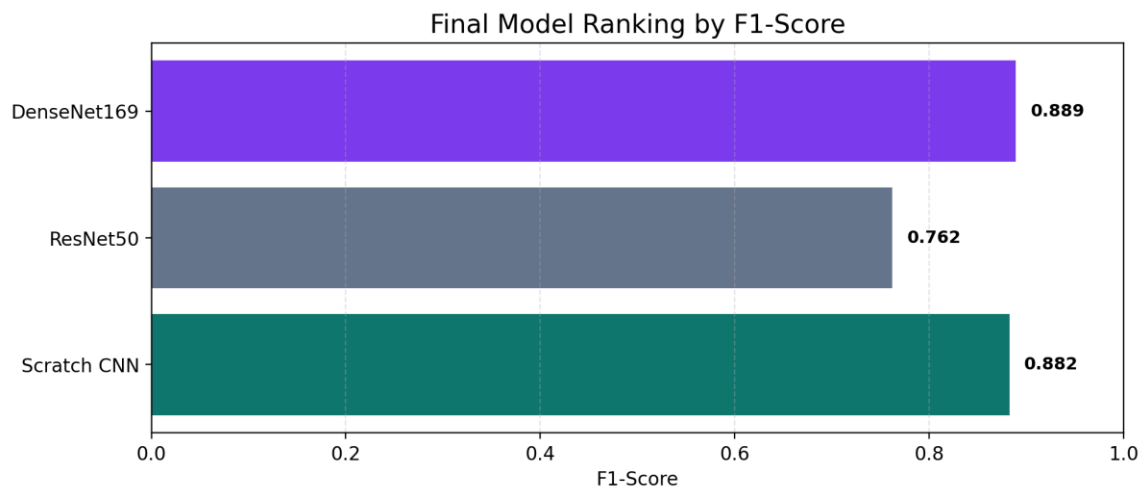
Model	Accuracy	Precision	Recall	F1-Score
Scratch CNN	0.964286	1.000000	0.789474	0.882353
ResNet50	0.910714	0.695652	0.842105	0.761905
DenseNet169	0.964286	0.941176	0.842105	0.888889

DenseNet169 achieved the strongest overall balance because it matched the best accuracy score while also achieving the highest F1-score. Scratch CNN achieved perfect precision, but its recall was lower, meaning it missed more actual cancer cases than DenseNet169.

Model comparison chart:



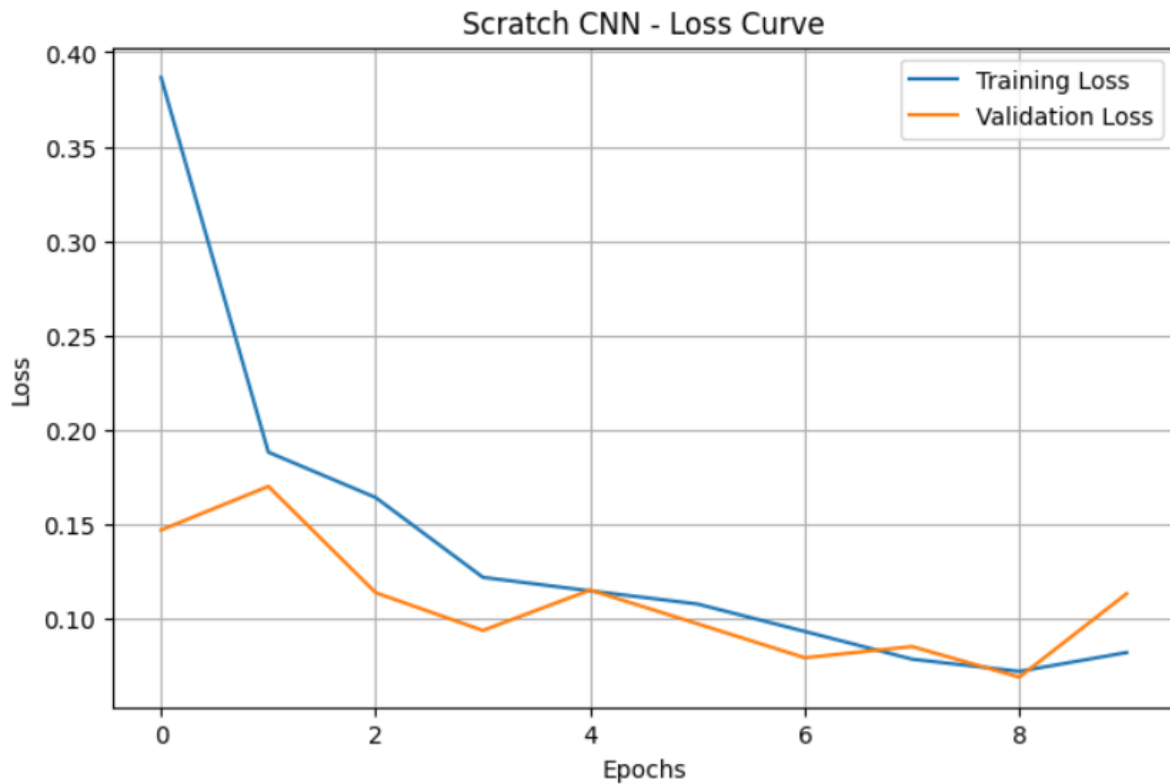
F1-score ranking:



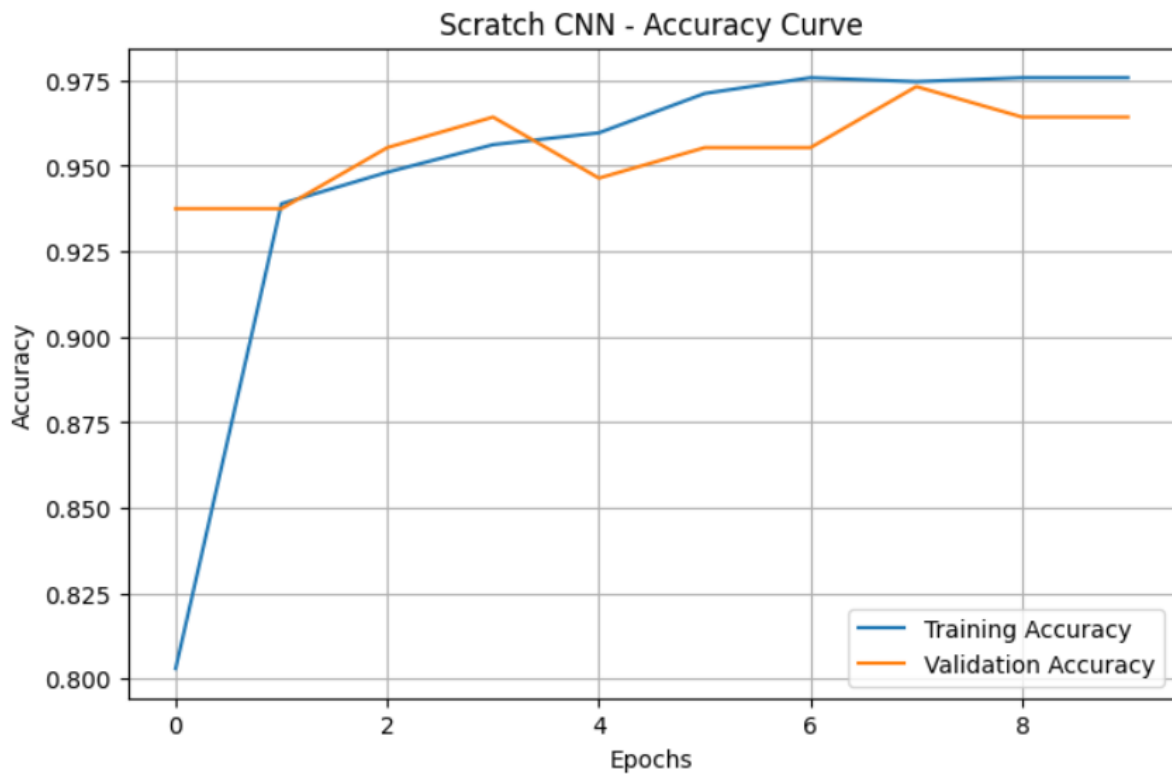
8. Training Curve Analysis

The Scratch CNN loss and accuracy curves show stable learning behavior over 10 epochs. Training loss decreased strongly, and validation loss remained close to training loss for most epochs. The final validation loss increased slightly, which may indicate early signs of overfitting.

Scratch CNN loss curve:



Scratch CNN accuracy curve:



The accuracy curve increased rapidly from the first epoch and stabilized near 96-97% validation accuracy. This suggests the model learned useful image features quickly. However, the slight validation loss rise near the final epoch suggests that early stopping could improve future training stability.

9. Model Saving and Deployment Readiness

Each trained model was saved in H5 format, allowing the models to be loaded later without retraining. A Streamlit dashboard can load any selected model, accept an uploaded image, preprocess it, and display the predicted class with confidence score.

```
model.save(r'D:\Downloads\Image Preprocessing Project\CNN_model.h5')
ResNet50_model.save(r'D:\Downloads\Image Preprocessing Project\ResNet50_model.h5')
DenseNet169_model.save(r'D:\Downloads\Image Preprocessing Project\DenseNet169_model.h5')
```

Dashboard Feature	Description
Image upload	The user uploads a JPG, PNG, BMP, or WEBP image.
Model selection	The user selects Scratch CNN, ResNet50, or DenseNet169.
Prediction	The selected model predicts Cancer or Non-Cancer.
Confidence display	The dashboard shows the predicted class and probability values.

10. Conclusion

This project successfully implemented and evaluated three deep learning models for breast cancer image classification. The evaluation pipeline included the required metrics and visualizations: accuracy, precision, recall, F1-score, confusion matrix, ROC/AUC curve, and loss/accuracy curves.

- Scratch CNN achieved high accuracy and perfect precision but lower recall.
- ResNet50 achieved the weakest overall result among the tested models.
- DenseNet169 achieved the best overall balance and the highest F1-score.
- DenseNet169 is recommended as the final model for this project based on the evaluation results.

11. Future Improvements

1. Add data augmentation such as rotation, zoom, flipping, and shifting.
2. Use early stopping to prevent overfitting when validation loss stops improving.
3. Fine-tune deeper layers of DenseNet169 or ResNet50 using a low learning rate.
4. Apply class weights if the dataset is imbalanced.
5. Evaluate on a larger and more diverse external test set.
6. Deploy the final model through a Streamlit dashboard for easier testing.